SESIÓN 04: CONTROL DE ACCESOS Y SISTEMA ANTI-TRAMPAS

Base de datos_ CHECK · Roles · Inyección SQL

Tres pilares

1. Integridad en el motor (CHECK): reglas que la BD aplica aunque se salte la app.
2. Roles con privilegio mínimo: cada perfil solo lo necesario.
3. Consultas parametrizadas: el texto del usuario nunca se interpreta como SQL.

1 · *Integridad en el motor (CHECK)*

El frontend se puede evadir. La base debe rechazar sola valores imposibles: nivel < 1, puntos negativos, precio nulo o stock bajo cero.

```
USE magdalena_ramos;
ALTER TABLE clientes
  ADD CONSTRAINT chk_clientes_nivel_minimo CHECK (nivel >= 1);
ALTER TABLE clientes
  ADD CONSTRAINT chk_clientes_puntos_no_negativos CHECK (puntos >= 0);
ALTER TABLE clientes
  ADD CONSTRAINT chk_clientes_email_valido CHECK (email LIKE '%@%.%');
ALTER TABLE ropa
  ADD CONSTRAINT chk_ropa_precio_valido CHECK (precio > 0.00);
ALTER TABLE ropa
  ADD CONSTRAINT chk_ropa_stock_no_negativo CHECK (stock >= 0);
```



Qué hace: cada CHECK registra una regla. Si un INSERT/UPDATE la viola, MySQL aborta (p. ej. ERROR 3819) y la fila no cambia.

2 · Roles: vendedor, supervisor, administrador

Tres roles en cadena: el supervisor hereda al vendedor; el administrador hereda al supervisor. Así no se repiten GRANTs y se respeta el mínimo privilegio.

Resumen: vendedor solo consulta; supervisor puede suspender clientes; administrador además modifica precios/stock y puede borrar.

3 · Procedimiento anti inyección SQL

Concatenar texto del usuario en el SQL permite ataques como ' OR '1'='1 o incluso ; DROP TABLE. La defensa: tratar el parámetro siempre como valor.

```
USE magdalena_ramos;
DROP PROCEDURE IF EXISTS buscar_cliente_seguro;
DELIMITER //
CREATE PROCEDURE buscar_cliente_seguro(IN p_nombre_usuario VARCHAR(50))
BEGIN
  SELECT cliente_id, nombre_usuario, email, telefono, direccion
  FROM clientes WHERE nombre_usuario = p_nombre_usuario;
END //
DELIMITER ;

CALL buscar_cliente_seguro('\' OR \'1\'=\'1');  -- debe dar 0 filas
CALL buscar_cliente_seguro('Magdalena_ramosAdmin');           -- búsqueda real
```

Lección: en el backend usar siempre prepared statements / placeholders. El mismo principio aplica con ORM o drivers con parámetros vinculados.

Conclusión de la sesión

Un sistema sólido valida en el motor (CHECK), limita quién hace qué (roles) y separa datos de código (procedimientos seguros). Esas capas reducen trampas, abusos y ataques.